



Faculty of Engineering and Technology Electrical and Computer
Engineering Department

**DIGITAL ELECTRONICS AND COMPUTER ORGANIZATION
LABORATORY ENCS2110**

Experiment no.10

Mustafa Altaweel-1221017

Sari Abd-Alghani-1220982

Instructor: Aziz Qaroush

Lab assistant: Raha Zabadi

Section: 6, Date: 21/5/2024

Abstract:

In this experiment we are going to explore how to design a simple computer using Verilog HDL, this simple computer is a very small computer that gives example of designing a simple CPU and a main memory with Verilog HDL notation.

Contents

Abstract:	2
1. Theory	4
1.1 Basic Computer Model – Von Neumann model	4
Figure 1.1.1: Von Neumann’s architecture	4
1.2 General Registers	5
1.3 Communication between Memory and Processing Unit	5
1.4 Generic CPU Instruction Cycle	5
1.5 Addressing modes.....	6
Figure 1.5.1: connection between memory and registers.....	7
2. Our Simple Computer	7
Figure 2.1: instruction format	8
3. Procedure.....	8
Figure 3.1: Verilog HDL implementation.....	9
Figure 3.2: waveform result	10
4. Conclusion	11
5. Resources	11

1. Theory

1.1 Basic Computer Model – Von Neumann model

Von Neumann computer architecture contains three main building blocks: central processing unit (CPU), main memory and input/output devices (I/O). These components are connected together using system bus. The most notable items in the CPU is registers: they can be manipulated directly by a computer program.

Function of the Von Neumann Component:

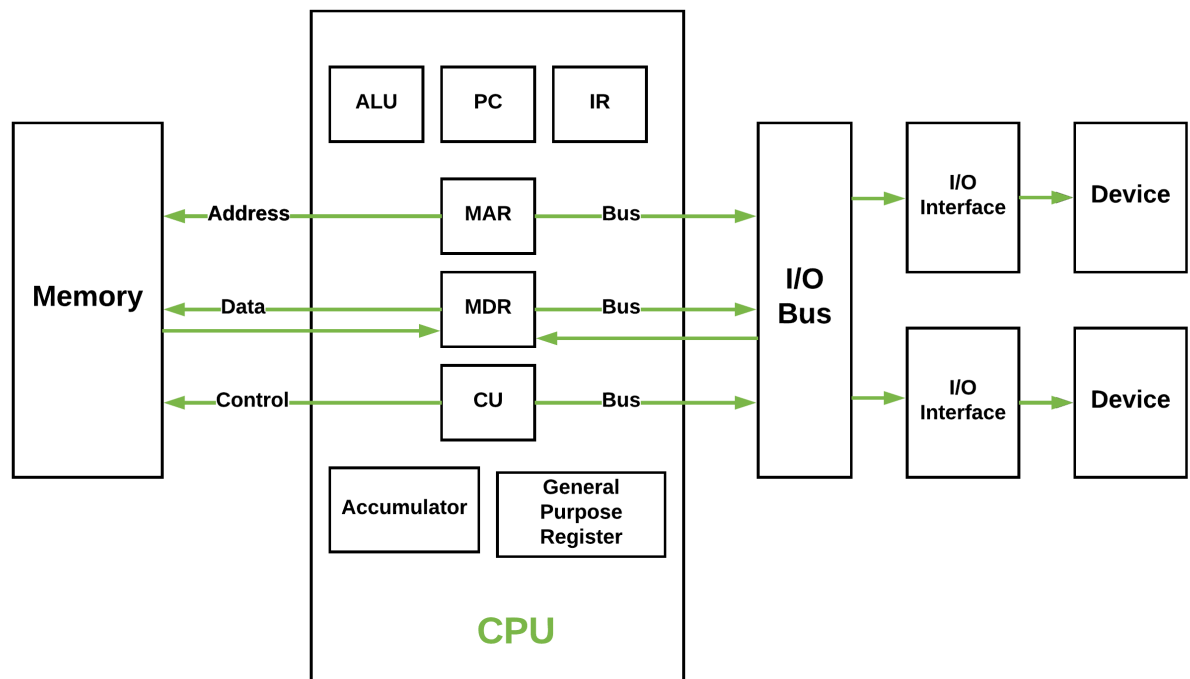
Processing Unit: Computation/Processing of Information

Input: Means of getting information into the computer. e.g., keyboard, mouse

Output: Means of getting information out of the computer. e.g., printer, monitor

Control Unit: Makes sure that all the other parts perform their tasks correctly and at the correct time.

Figure 1.1.1: Von Neumann's architecture



1.2 General Registers

1. Accumulator (AC): it's the main operand register of ALU and is used to store the result generated by ALU.
2. Data register (MDR): acts as a buffer between CPU and main memory. It is used as an input operand register with the AC.
3. Instruction register (IR): holds the opcode of the current instruction.
4. Address register (MAR): holds the address of the memory location in which the operand resides.
5. Program counter (PC): holds the address of the next instruction to be fetched/executed.

Additional addressable registers can be provided for storing operands and address. This can be viewed as replacing the single accumulator by a set of registers. If the registers are used for many purposes, the resulting computer is said to have general register organization. In the case of processor registers, a register is selected by the multiplexers that form the buses.

1.3 Communication between Memory and Processing Unit

Communication between memory and processing unit consists of two registers: Memory Address Register (MAR) and Memory Data Register (MDR).

To read,

1. The address of the location is put in MAR.
2. The memory is enabled for a read.
3. The value is put in MDR by the memory.

To write,

1. The address of the location is put in MAR.
2. The data is put in MDR.
3. The Write Enable signal is asserted.
4. The value in MDR is written to the location specified.

1.4 Generic CPU Instruction Cycle

1. **Instruction fetch:** read instruction from address in PC and place in IR.
2. **Instruction decode:** Hardware determines what the opcode/function is and determines which registers or memory addresses contain the operands.
3. **Operand fetch (from memory if necessary):** if any operands are memory addresses, memory read cycles to read them into CPU registers, If an operand is in memory,

not a register, then the memory address of the operand is known as the *effective address* (EA). The fetching of an operand is basically denoted as (Register $\leftarrow$ Memory [EA]). On today's Computers, CPUs are much faster than main memory so it may take some several CPU cycles.

4. **Execute:** Perform the function of the instruction, if the operand is logical or arithmetic, CPU employs the ALU to store the result on an output register, this is the only stage of the instruction where the user can see the result. Everything else is overhead required to make the execute stage happen. One of the major goals of CPU design is to eliminate overhead and spend a higher percentage of the time in the execute stage.
5. **Store result in memory if necessary:** if destination is a memory address, initiate a write cycle to transfer data from CPU to memory. It depends on the situation whether the CPU wait or don't for the operation to be completed. If the next instruction doesn't use a memory address then it can move to the next instruction while carrying data saved from write operation.

Example of a full instruction cycle using memory address:

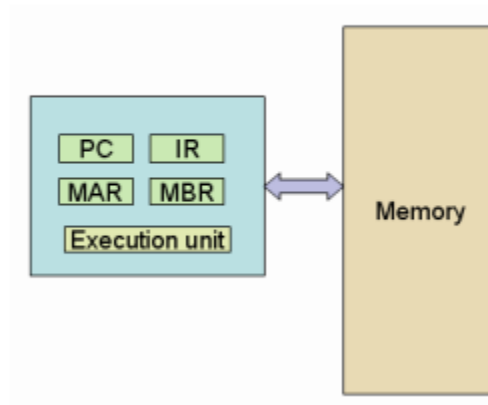
Multiply X, Y:

1. **Fetching:** fetches the instruction code from memory (PC register).
2. **Decoding:** this is to let the CPU know that it's a multiply operation and X, Y are the operands.
3. **Fetching:** fetches X, Y from memory.
4. **Execute:** multiplies X by Y and stores the result in a CPU register/
5. **Storing:** stores the value in CPU register to a memory location.

1.5 Addressing modes

The term **addressing modes** refers to the way in which the operand of an instruction is specified. Information contained in the instruction code is the value of the operand or the address of the result/operand.

Figure 1.5.1: connection between memory and registers.



Addressing modes are divided into:

1. **Immediate addressing:** where you directly includes the data to be acted as a part of the instruction.
2. **Direct addressing:** where the address part of the instruction is actually a memory location that includes the operand.

2. Our Simple Computer

Our “simple-comp” contains a 16 bits addressable memory with size of 128bytes. The memory is synchronous to the CPU, and the CPU can read or write a word (or cell) in single clock period. The memory can only be accessed through the **MAR** register and **MBR** register, to read from memory, we use this procedure:

- **MBR** $\leftarrow$ memory[**MAR**]

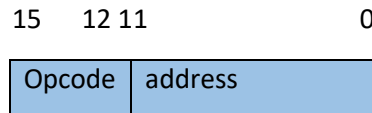
And to write to memory we use procedure:

- Memory[**MAR**] $\leftarrow$ **MBR**

These are the properties of our simple-comp:

1. CPU has three registers only: accumulator (**AC**), program counter (**PC**) and an instruction register (**IR**).
2. Simple-comp has only four instructions: **Load**, **Store**, **Add** and **Jump**.
3. The size of all instructions is 16 bits; all the instructions are single address instructions and access a word in memory.

Figure 2.1: instruction format



The opcodes are:

Opcode	Instruction	Description
0011	Load M(X)	Loads the contents of memory location M(X) into the accumulator.
1011	Store M(X)	Stores the contents of the accumulator in memory location M(X).
0111	Add M(X)	Adds the contents of memory location M(X) to the contents of the accumulator.
0001	Jump M(X)	Let the PC value be the memory location M(X) and fetch that instruction.

3. Procedure

We implemented the information above in Quartus II software by using Verilog HDL file, then we simulated the file by making some instructions above.

Figure 3.1: Verilog HDL implementation

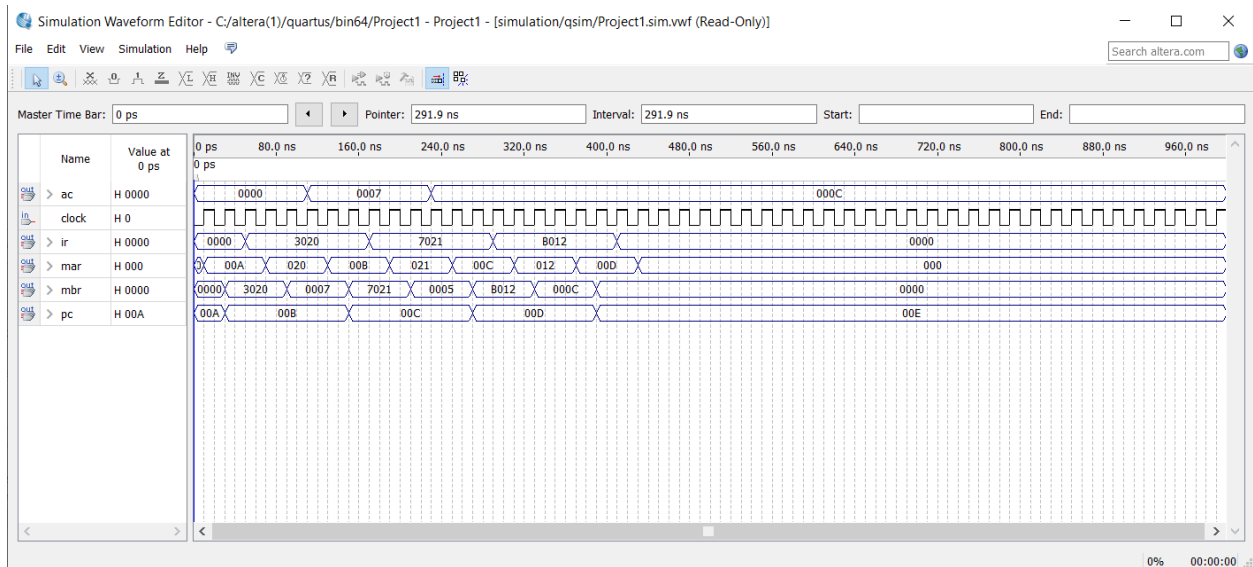
```

4 //define the output, PC:program counter,
5 //IR:instruction register, MBR:memory buffer register, AC:accumulator,
6 //MAR:memory address register, outdx: output devices
7 output pc,ir,mbr,ac,mr;
8 //define the registers
9 reg [15:0] ir,mbr,ac;
10 reg [11:0] pc,mr;
11 reg [15:0] memory [0:63]; //memory of 64 words, each word is 2 bytes
12 reg [2:0] state;
13 //provide opcodes for the instructions
14 parameter load = 4'b0011,store=4'b1011,add=4'b0111,jump=4'b0001;
15
16 initial begin
17 //instruction memory
18 memory[10]=16'h3020; //load location 32 from memory to AC
19 memory[11]=16'h7021; //add AC with location 33 and save to AC
20 memory[12]=16'hB012; //store AC in location 12
21
22 //data memory
23 memory[32]=16'd7; //store number in location 12 in memory
24 memory[33]=16'd5; //store number in location 13 in memory
25 pc=10; //start with PC = 10, so first instruction will be the one that in location 10 (16'h300a)
26 state=0; //state in indicate to each state : instruction fetch, instruction decode, operand fetch, execution
27 end
28
29
30 always @ (posedge clock) begin // loop one time for each clock rising edge
31 always @ (posedge clock) begin // loop one time for each clock rising edge
32 case (state)
33 0:begin //initialization of MAR to get the instruction from memory
34 mar <= pc; //store PC to MAR because when we want to access the memory, we should store the address in MAR
35 state=1; //go to next step next clock
36 end
37 1:begin //instruction fetch
38 mbr <= memory[mr]; //store the instruction in IR
39 pc <= pc+1; //go one instruction a head
40 state=2;
41 end
42 2:begin //instruction fetch
43 ir <= mbr; //store the instruction in IR
44 state=3;
45 end
46 3:begin //instruction decode
47 mar <= ir[11:0]; //store the address in MAR to access the memory in the next step in case the instruction is Load or Store
48 state=4;
49 end
50 4:begin //operand fetch
51
52 4:begin //operand fetch
53 state=5;
54 case (ir[15:12]) // fill the value of MBR with the correct pattern in each operation type
55 load : mbr <= memory[mr];
56 add : mbr <= memory[mr];
57 store: mbr <= ac;
58 jump : mbr <= mar;
59 endcase
60 end
61 5:begin //execution
62 if (ir[15:12]==4'h7)begin //addition
63 ac <= ac+mbr;
64 state=0;
65 end
66 else if (ir[15:12]==4'h3)begin //load
67 ac <= mbr;
68 state=0;
69 end
70 else if (ir[15:12]==4'hb)begin //store
71 memory[mr] <= mbr;
72 state=0;
73 end
74 else if (ir[15:12]==4'h1)begin //jump
75 pc <= mbr;
76 state=0;
77 end
78 endcase
79 end
80 endmodule

```

And this is how the waveform came as a result of the three instructions:

Figure 3.2: waveform result



As we can see the simulation, we can tell these notes:

1. **AC** had the loaded value and the sum value
2. **IR** had all the instructions in memory location 10,11,12
3. **MAR** had all memory locations we went through
4. **MBR** had the values of these memory locations
5. **PC** kept counting from 10 until we finished the 3 instruction which is 13.

4. Conclusion

We learned in this experiment about Von Neumann architecture and the components of it which includes **processing unit** that have a variety of registers such as **PC, AC, MDR, MAR**, etc., then we knew how an instruction is being executed in CPU and what is the cycle and steps of it like **fetching, decoding**, etc., Digital Electronics and Computer Organization Lab, after that, we learned the instruction format and what is the opcode and what is a memory address, finally we implemented all this information to make a simple computer using Verilog HDL to make a simple CPU that makes the instructions **Load, Store, Add, Jump**.

5. Resources

All figures and laws had been taken from Birzeit University official Digital Electronics and Computer Organization Lab manual.